

Лабораторная работа №2

«Реализация геопоиска по Lat/Lng по карте»

Задание:

Что должна уметь система:

- Помещать географически расположенные объекты
- Искать объекты по координатам — очевидно, не по точным (тут бы хватило любого KV), а близким

В работе также требуется:

- Написать бенчмарки, в которых показать либо сравнение алгоритма с разумными аналогами, либо изменение времени работы алгоритма в зависимости от размера данных (размер БД, запрашиваемые ключи и точность)

Реализация:

В лабораторной работе реализован механизм геопоиска, основанный на трех структурах данных:

- 1) KDTree - двоичное дерево, рекурсивно разбивающее пространство по одной из координат на каждом уровне. Обеспечивает быстрый поиск k ближайших соседей и поиск в радиусе за счёт отсечения ветвей, не пересекающих область запроса. Вставка выполняется за $O(\log n)$ в сбалансированном случае, но при несбалансированных данных может вырождаться в $O(n)$.
- 2) RTree - иерархическая структура, группирующая объекты в минимальные ограничивающие прямоугольники (MBR). Узлы могут содержать переменное количество записей, что обеспечивает адаптивность к динамическим данным. Поиск требует обхода узлов, чьи MBR пересекают область запроса. Качество поиска зависит от минимизации перекрытия MBR.
- 3) OctoTree - разбивает пространство на четыре квадранта, рекурсивно деля области до достижения порога точек в узле. Позволяет быстро локализовать точки в заданном радиусе за счёт отсечения квадрантов, не пересекающих круг. Эффективно при равномерном распределении данных, но может создавать глубокие ветви при неравномерном распределении.

Тестирование:

Замеры бенчмарков осуществлялись с помощью JMH:

```
# VM version: JDK 25.0.1, OpenJDK 64-Bit Server VM, 25.0.1+8-27
# VM invoker: C:\Users\dmp12\jdk\openjdk-25.0.1\bin\java.exe
# VM options: -javaagent:D:\IntelliJ IDEA Community Edition 2025.2.1\lib\idea_rt.jar=2613 -
Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8
# Blackhole mode: compiler (auto-detected, use -Djmh.blackhole.autoDetect=false to disable)
# Warmup: 5 iterations, 1 s each
# Measurement: 20 iterations, 1 s each
# Timeout: 10 min per iteration
# Threads: 1 thread, will synchronize iterations
# Benchmark mode: Average time, time/op
```

Для каждого алгоритма сделаны отдельные benchmark-методы (построение, поиск, вставка/чтение). Размеры входных данных задаются через @Param (10000, 100000, 500000, 1000000). Результаты автоматически сохраняются в CSV (target/jmh-results).

Все замеры проводились на ноутбуке со следующими характеристиками:

- Операционная система – Windows 10 (64-разрядная операционная система, процессор x64)
- Процессор: AMD Ryzen 7 4700U with Radeon Graphics 2.00 GHz, 8 ядер
- Оперативная память: 16,0 ГБ
- Видеоадаптер: AMD Radeon(TM) Graphics (2 GB)

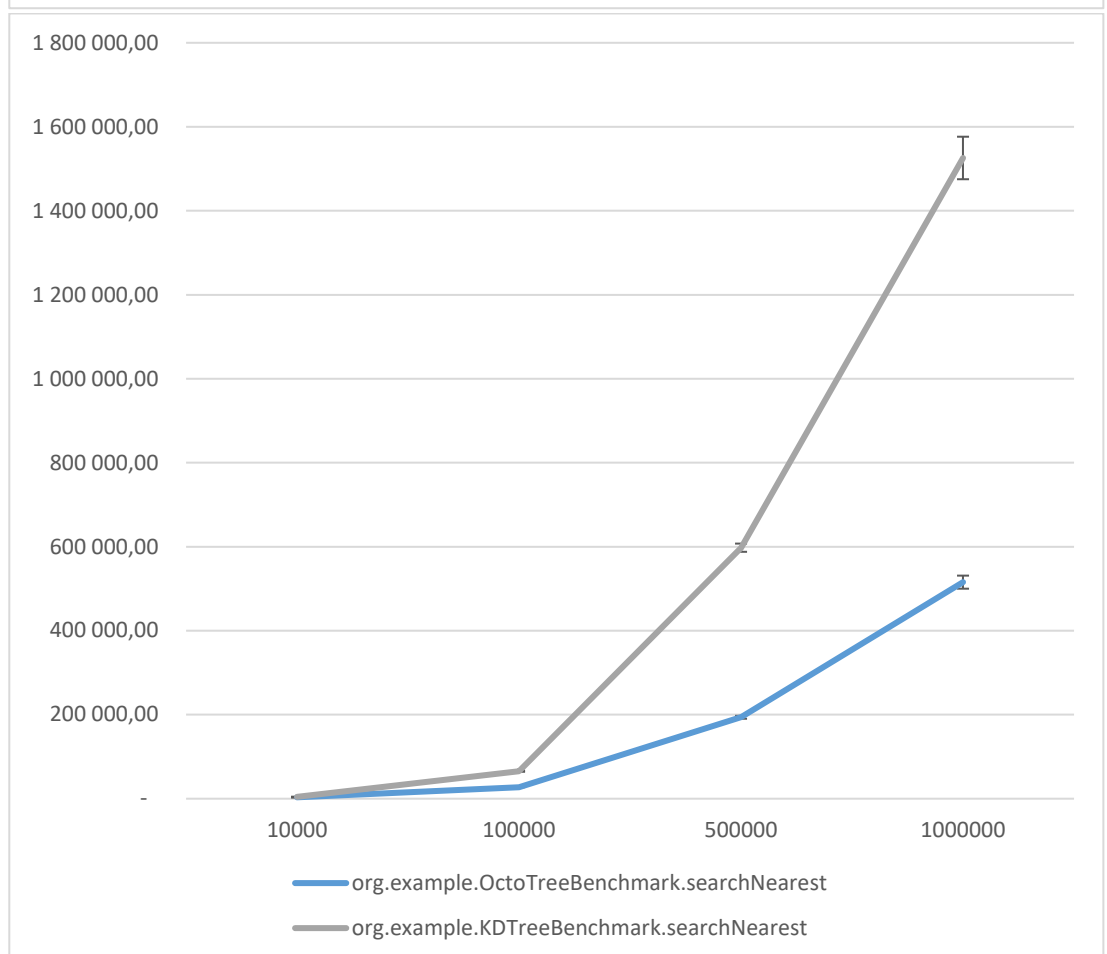
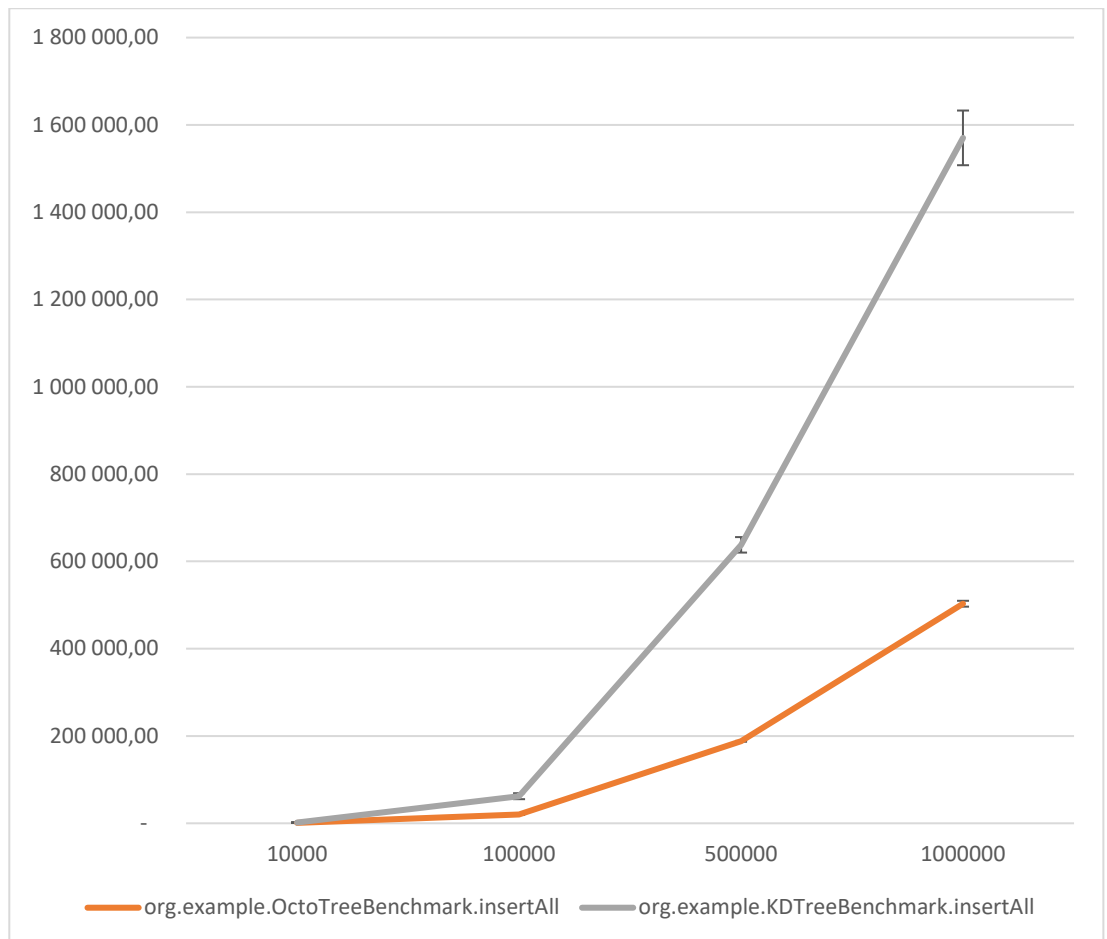
Во время проведения замеров ноутбук был подключен к сети.

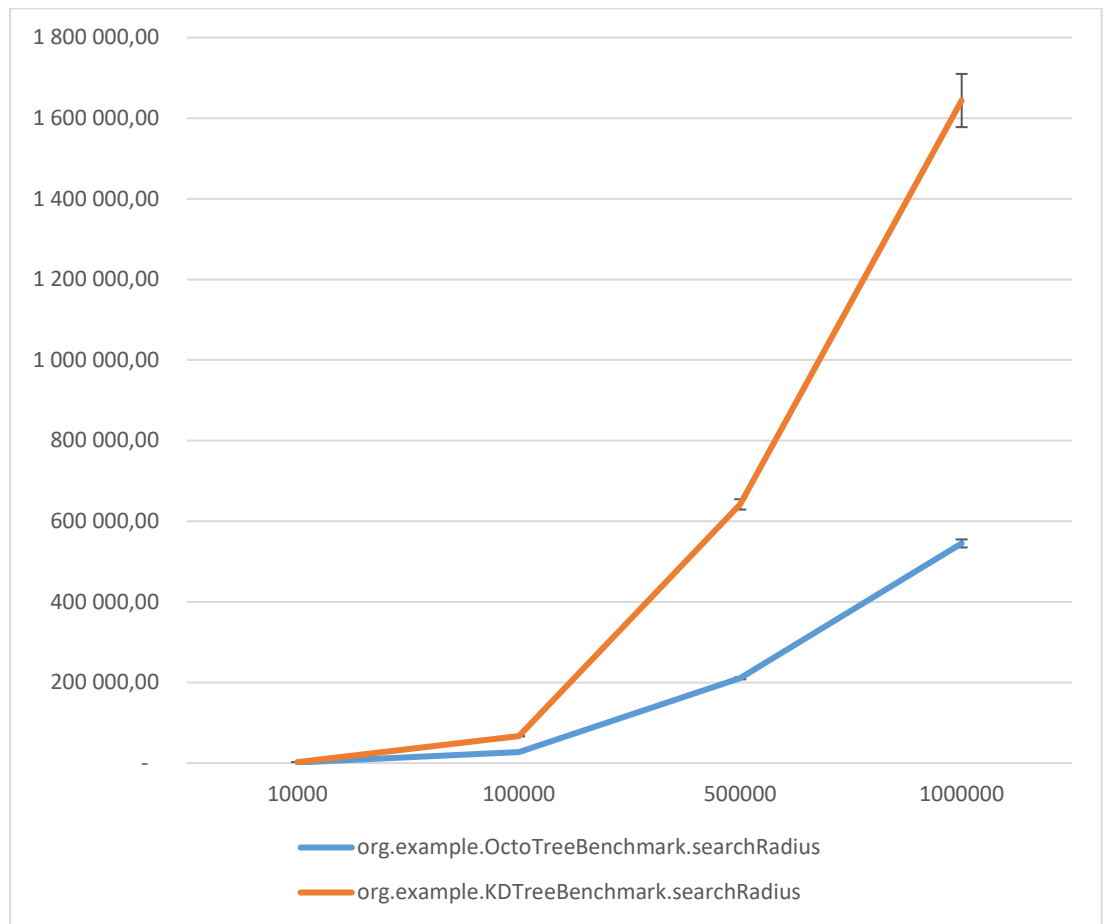
1) KDTree

Benchmark	Mode	Threads	Samples	Score	Score Error (99,9%)	Unit	Param: size
insertAll	avgt	1	20	1 762,42	63,89	us/op	10000
insertAll	avgt	1	20	62 131,61	6 836,86	us/op	100000
insertAll	avgt	1	20	637 956,34	17 738,42	us/op	500000
insertAll	avgt	1	20	1 570 270,86	62 640,50	us/op	1000000
searchNearest	avgt	1	20	4 452,36	43,44	us/op	10000
searchNearest	avgt	1	20	65 475,71	1 317,52	us/op	100000
searchNearest	avgt	1	20	597 717,80	9 769,42	us/op	500000
searchNearest	avgt	1	20	1 525 751,95	50 632,76	us/op	1000000
searchRadius	avgt	1	20	2 449,04	9,41	us/op	10000
searchRadius	avgt	1	20	67 259,98	963,82	us/op	100000
searchRadius	avgt	1	20	641 916,75	12 828,20	us/op	500000
searchRadius	avgt	1	20	1 643 910,54	66 044,61	us/op	1000000

2) OctoTree

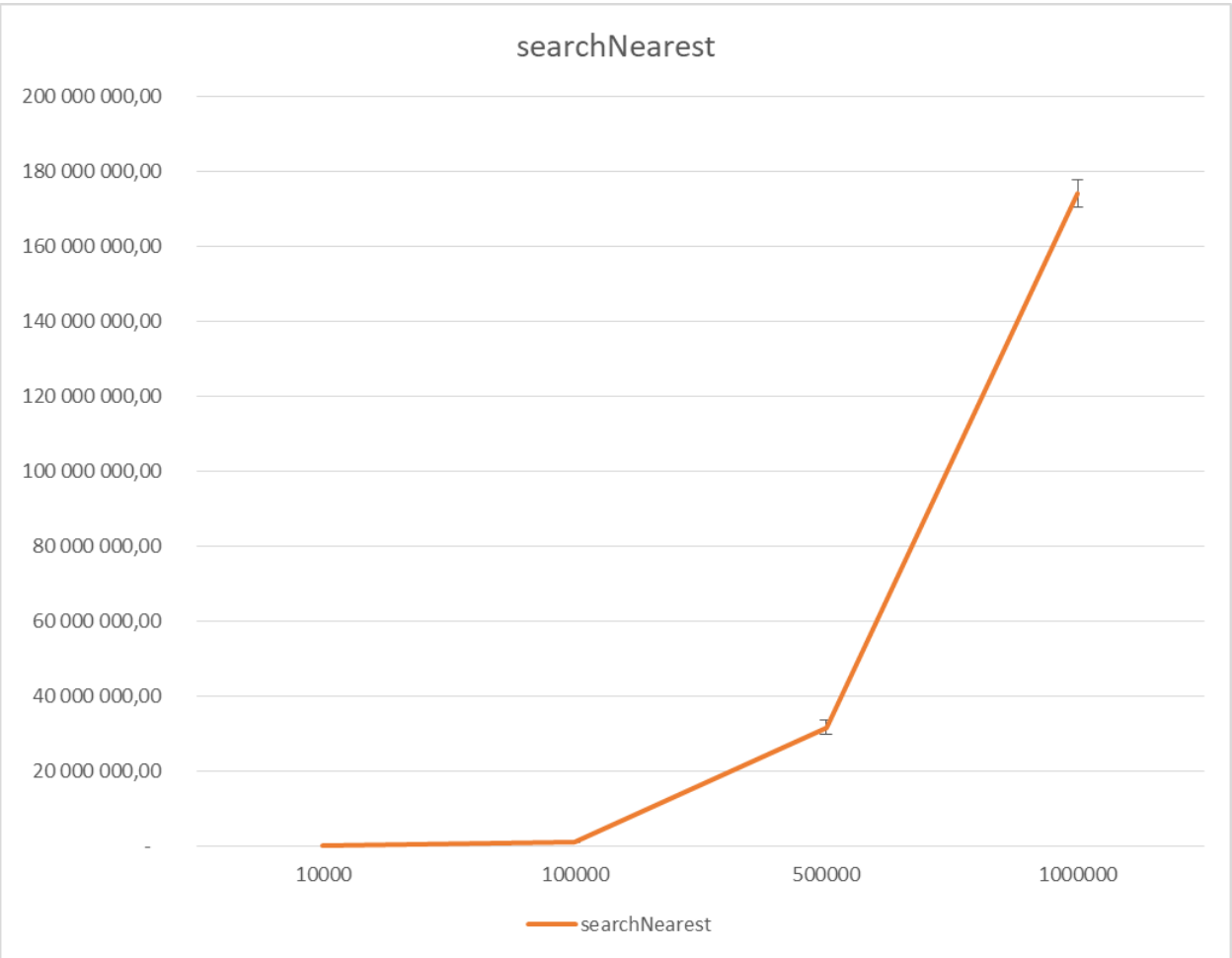
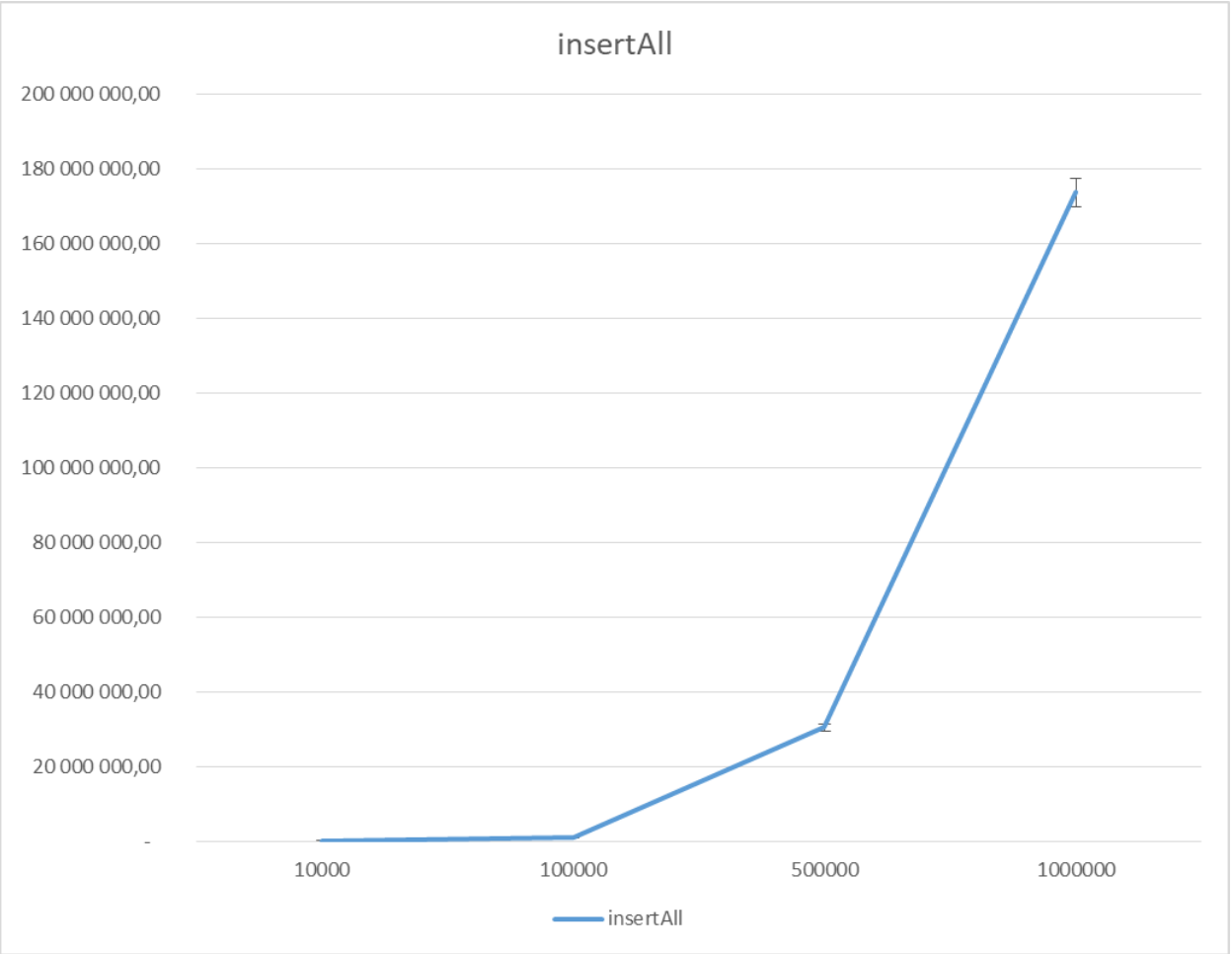
Benchmark	Mode	Threads	Samples	Score	Score Error (99,9%)	Unit	Param: size
insertAll	avgt	1	20	993,97	8,94	us/op	10000
insertAll	avgt	1	20	19 954,58	246,35	us/op	100000
insertAll	avgt	1	20	187 855,22	962,70	us/op	500000
insertAll	avgt	1	20	503 063,35	6 714,66	us/op	1000000
searchNearest	avgt	1	20	3 327,24	14,37	us/op	10000
searchNearest	avgt	1	20	27 646,12	286,15	us/op	100000
searchNearest	avgt	1	20	193 933,63	3 493,18	us/op	500000
searchNearest	avgt	1	20	515 670,26	15 566,19	us/op	1000000
searchRadius	avgt	1	20	1 541,52	39,50	us/op	10000
searchRadius	avgt	1	20	27 302,80	916,79	us/op	100000
searchRadius	avgt	1	20	210 489,22	2 754,30	us/op	500000
searchRadius	avgt	1	20	544 973,58	9 964,70	us/op	1000000

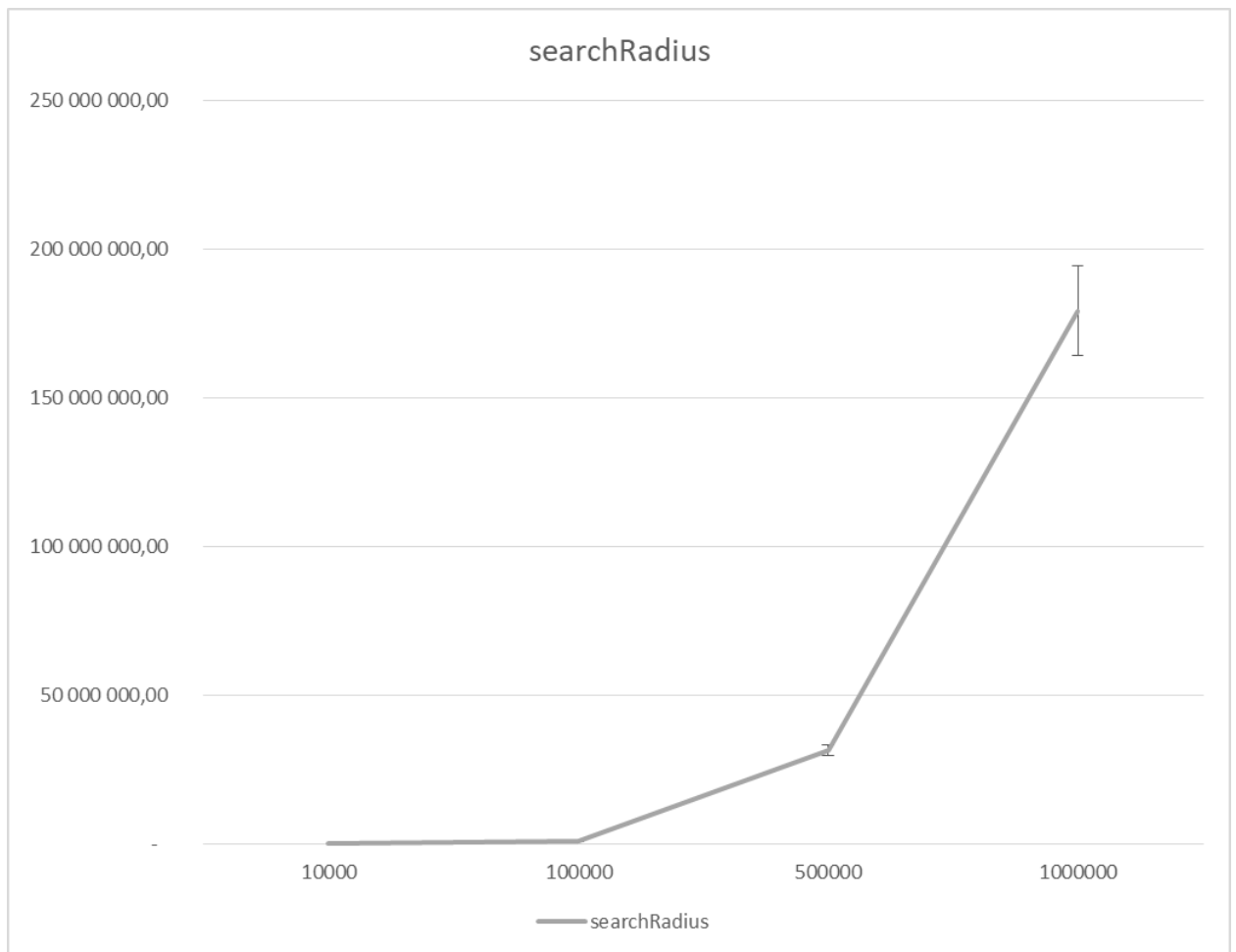




3) RTree

Benchmark	Mode	Threads	Samples	Score	Score Error (99,9%)	Unit	Param: size
insertAll	avgt	1	20	23 515,66	749,30	us/op	10000
insertAll	avgt	1	20	942 075,51	8 988,25	us/op	100000
insertAll	avgt	1	20	30 343 136,28	1 071 689,96	us/op	500000
insertAll	avgt	1	20	173 534 037,93	3 847 482,19	us/op	1000000
searchNearest	avgt	1	20	34 229,54	443,72	us/op	10000
searchNearest	avgt	1	20	969 289,30	8 133,95	us/op	100000
searchNearest	avgt	1	20	31 658 165,17	1 887 536,16	us/op	500000
searchNearest	avgt	1	20	174 132 780,42	3 701 001,20	us/op	1000000
searchRadius	avgt	1	20	24 089,53	144,85	us/op	10000
searchRadius	avgt	1	20	938 085,28	5 645,84	us/op	100000
searchRadius	avgt	1	20	31 495 813,91	1 829 791,42	us/op	500000
searchRadius	avgt	1	20	179 304 960,85	15 059 258,46	us/op	1000000





Выводы:

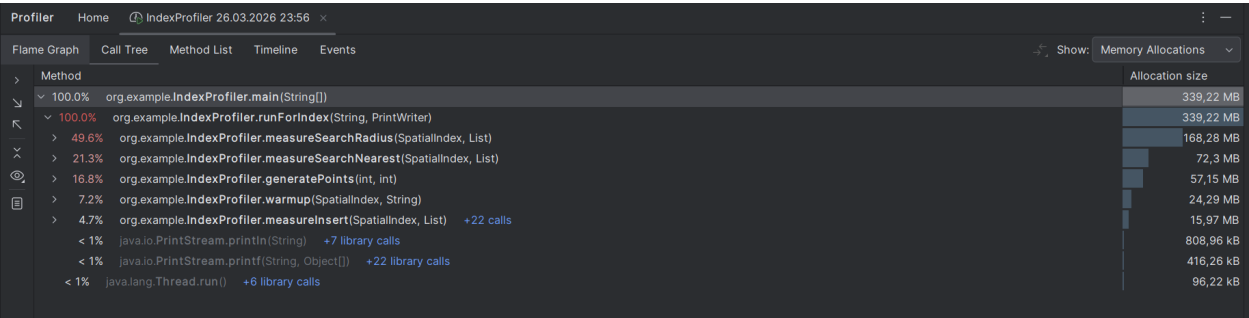
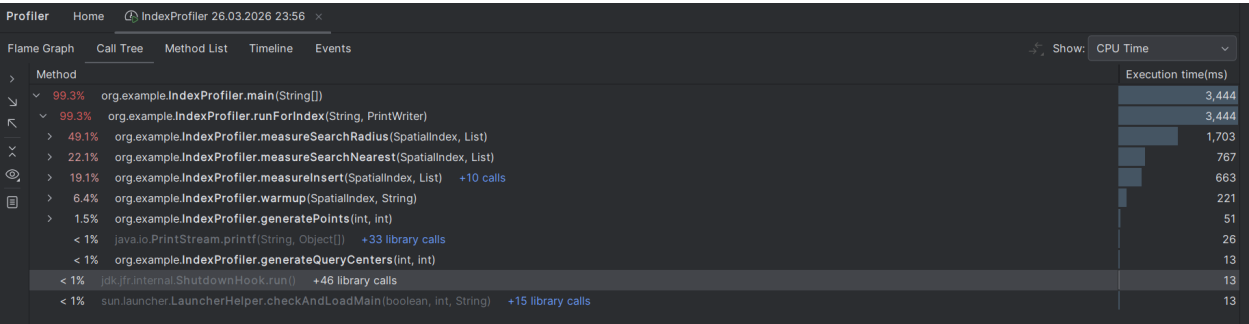
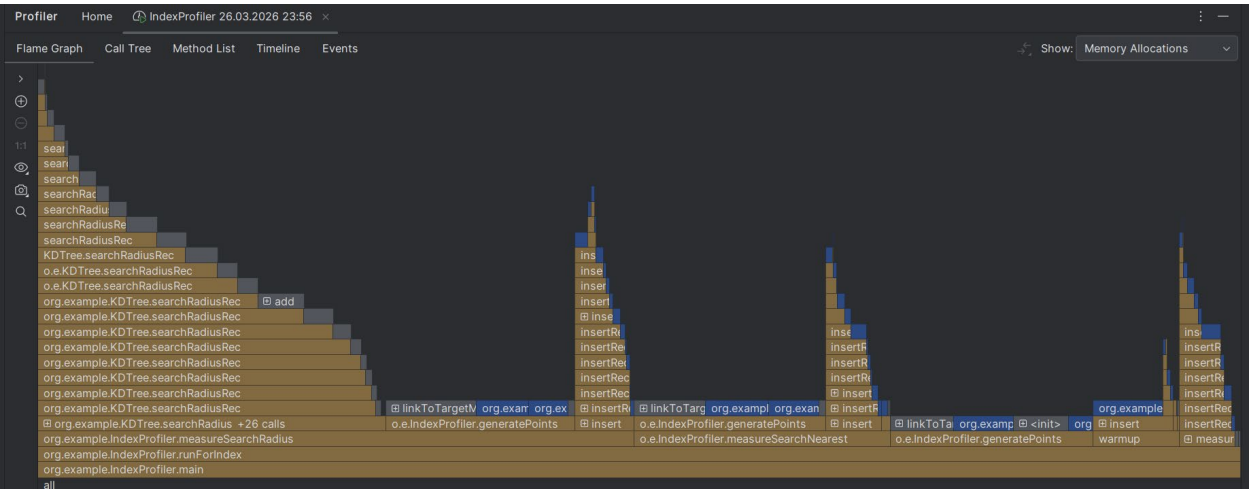
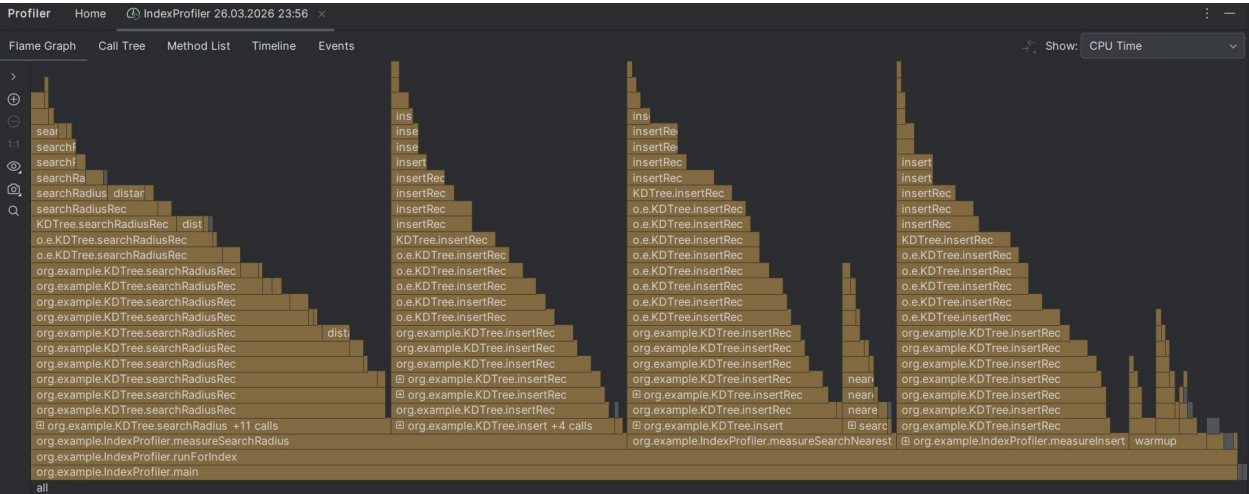
- insertAll – Вставка - OctoTree демонстрирует наиболее быстрое построение: для 1 млн точек примерно 0,5 млн us/op, что почти вдвое быстрее KDTree. Это объясняется тем, что вставка ограничена локальными разбиениями и не требует рекурсивного поиска по всем уровням для каждой точки. RTree имеет наименьшую производительность вставки.
- searchRadius - Поиск в радиусе - KDTree и OctoTree показывают схожее время поиска, с небольшим преимуществом OctoTree. OctoTree вероятно быстрее благодаря жёсткому отсечению по квадрантам, в то время как KDTree может посещать больше ветвей из-за неоптимального разбиения. RTree также сильно отстает в данном тесте.
- searchNearest - Поиск ближайших объектов - Картина аналогична предыдущему тесту. KDTree и OctoTree демонстрируют близкие результаты, но OctoTree снова опережает KDTree. RTree сильно отстает и имеет большую погрешность.

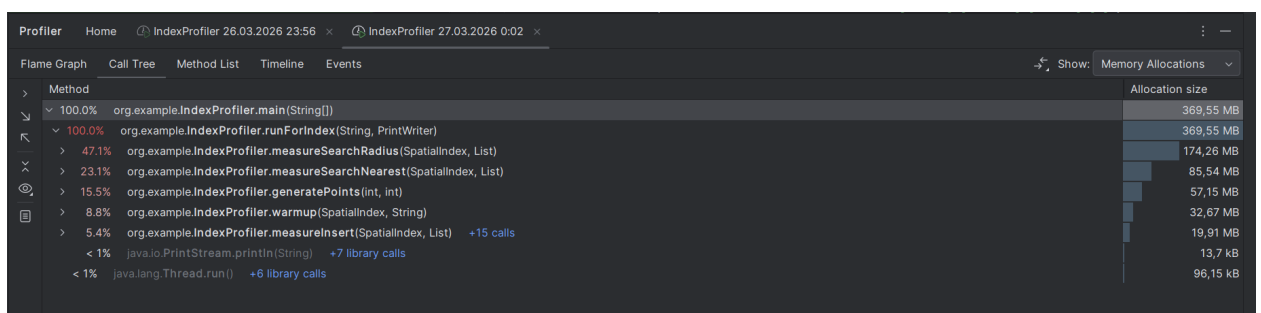
Таким образом, структура OctoTree показала наилучшее время выполнения как для вставки, так и для поиска (в 2–3 раза быстрее KDTree и на порядки быстрее RTree) в текущей реализации. По результатам тестирования она является оптимальным выбором.

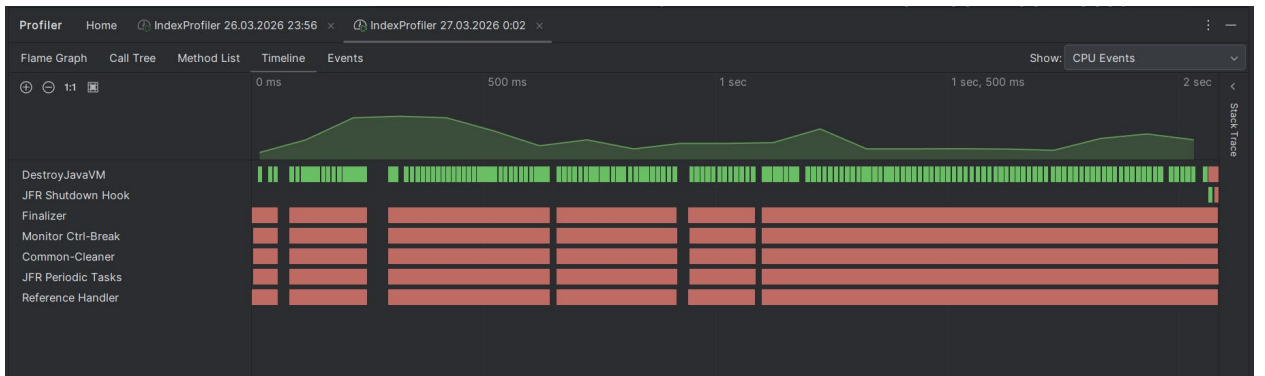
Профайлинг:

Профайлинг осуществлялся с помощью IntelliJ Profiler для 500000 точек.

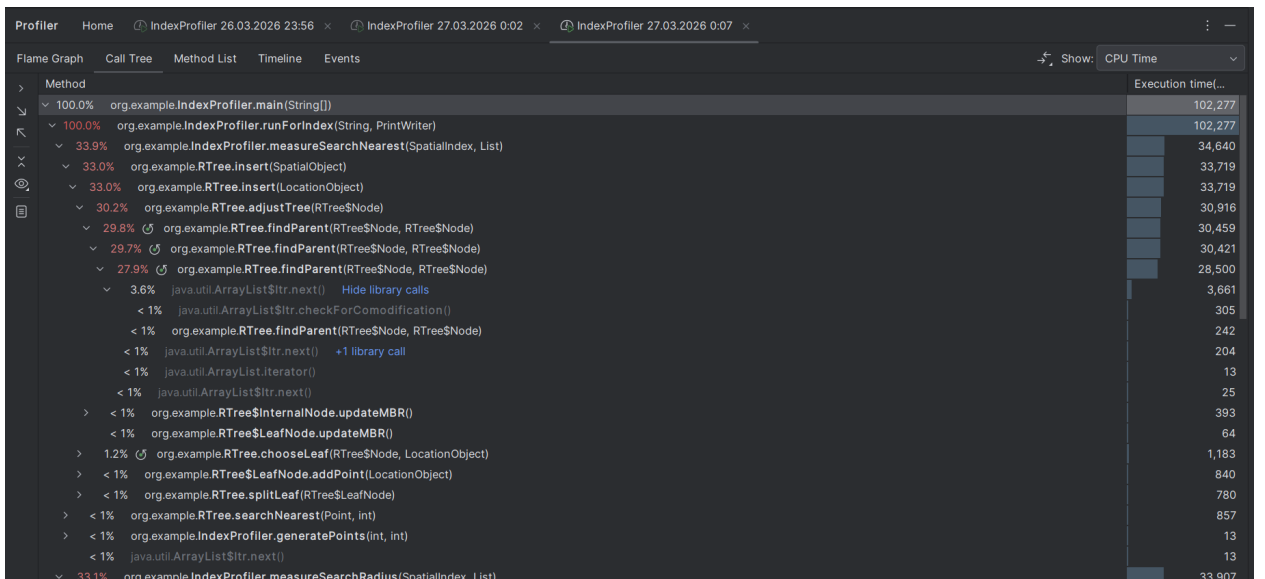
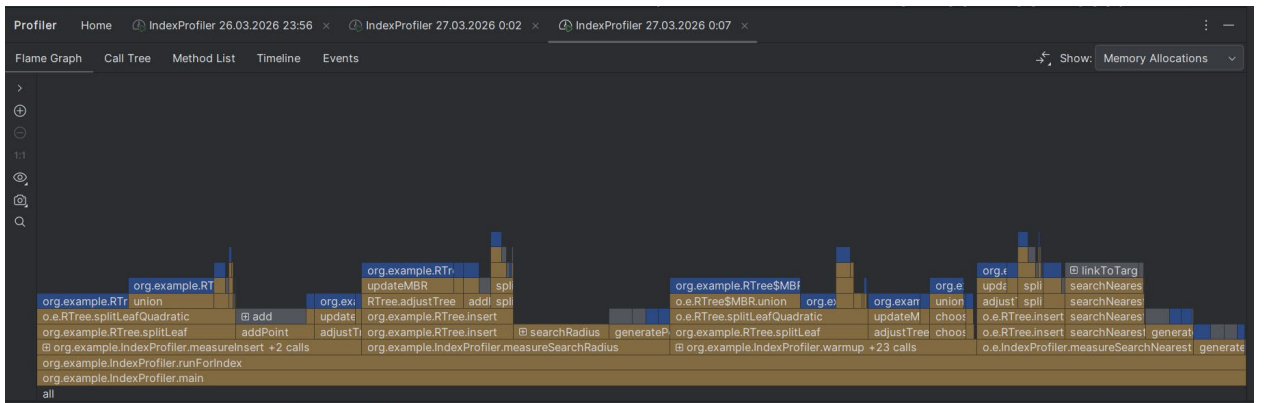
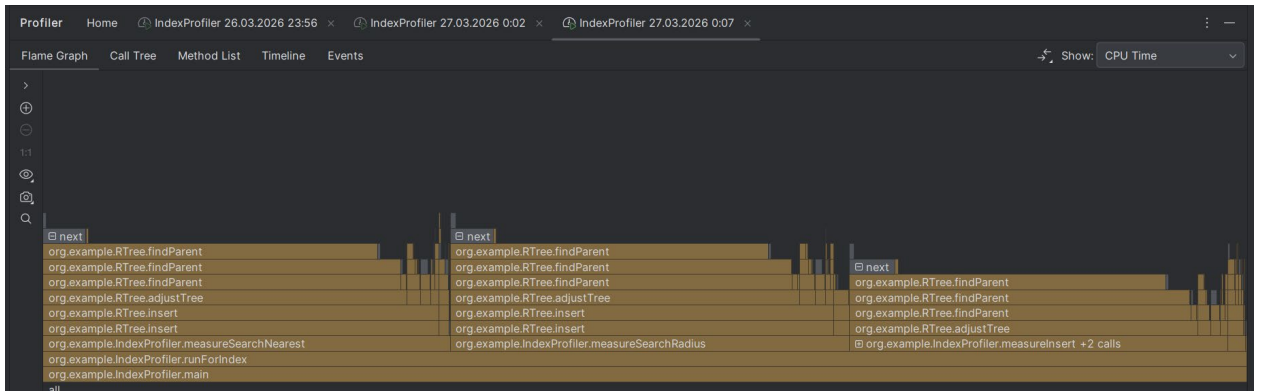
1) KDTree







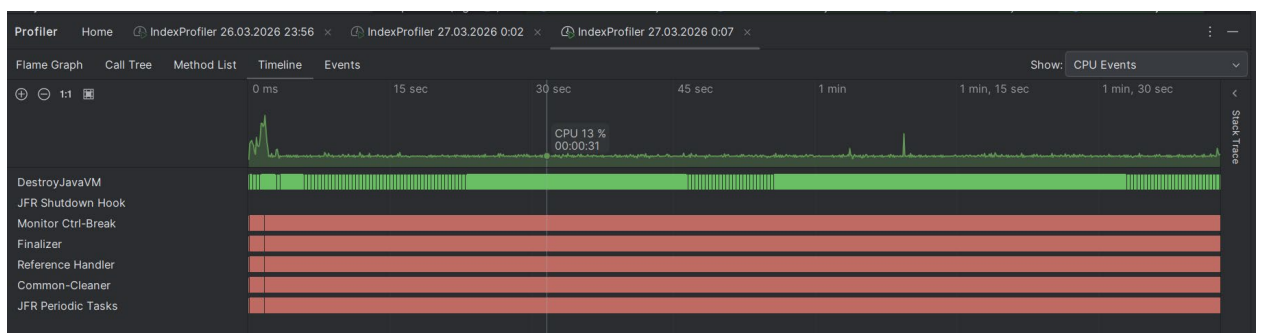
3) RTree



33.1%	org.example.IndexProfiler.measureSearchRadius(SpatialIndex, List)	33,907
31.9%	org.example.RTree.insert(SpatialObject)	32,581
31.9%	org.example.RTree.insert(LocationObject)	32,581
29.0%	org.example.RTree.adjustTree(RTreeNode)	29,678
28.4%	org.example.RTree.findParent(RTreeNode, RTreeNode)	29,043
28.3%	org.example.RTree.findParent(RTreeNode, RTreeNode)	28,941
26.4%	org.example.RTree.findParent(RTreeNode, RTreeNode)	27,009
3.6%	java.util.ArrayList\$Itr.next()	3,648
< 1%	java.util.ArrayList\$Itr.checkForComodification()	470
< 1%	org.example.RTree.findParent(RTreeNode, RTreeNode)	254
< 1%	java.util.ArrayList\$Itr.next()	204
< 1%	java.util.ArrayList\$Itr.next()	13
> < 1%	org.example.RTree\$InternalNode.updateMBR()	456
> < 1%	org.example.RTree\$LeafNode.updateMBR()	166
> 1.3%	org.example.RTree.chooseLeaf(RTreeNode, LocationObject)	1,287
> < 1%	org.example.RTree\$LeafNode.addPoint(LocationObject)	903
> < 1%	org.example.RTree.splitLeaf(RTreeNode)	713
> 1.3%	org.example.RTree.searchRadius(Point, double)	1,288
< 1%	org.example.IndexProfiler.generatePoints(int, int)	13

31.4%	org.example.IndexProfiler.measureInsert(SpatialIndex, List)	+2 calls	32,157
29.0%	org.example.RTree.adjustTree(RTreeNode)		29,614
28.4%	org.example.RTree.findParent(RTreeNode, RTreeNode)		29,031
28.3%	org.example.RTree.findParent(RTreeNode, RTreeNode)		28,903
26.3%	org.example.RTree.findParent(RTreeNode, RTreeNode)		26,870
3.8%	java.util.ArrayList\$Itr.next()	Hide library calls	3,876
< 1%	java.util.ArrayList\$Itr.checkForComodification()		394
< 1%	org.example.RTree.findParent(RTreeNode, RTreeNode)		267
< 1%	java.util.ArrayList\$Itr.next()	+1 library call	242
< 1%	java.util.ArrayList\$Itr.next()		13
> < 1%	org.example.RTree\$InternalNode.updateMBR()		456
> < 1%	org.example.RTree\$LeafNode.updateMBR()		102
> 1.0%	org.example.RTree.chooseLeaf(RTreeNode, LocationObject)		1,073
> < 1%	org.example.RTree\$LeafNode.addPoint(LocationObject)		816
> < 1%	org.example.RTree.splitLeaf(RTreeNode)		628
> 1.5%	org.example.IndexProfiler.warmup(SpatialIndex, String)		1,496
> < 1%	org.example.IndexProfiler.generatePoints(int, int)		77
< 1%	jdk.internal.ShutdownHook.run()	+40 library calls	13

Profiler			Home	IndexProfiler 26.03.2026 23:56	IndexProfiler 27.03.2026 0:02	IndexProfiler 27.03.2026 0:07
Flame Graph						
Call Tree						
Method List						
Timeline						
Events						
Show: Memory Allocations						
Method						Allocation size
100.0%	org.example.IndexProfiler.main(String[])					1,36 GB
100.0%	org.example.IndexProfiler.runForIndex(String, PrintWriter)					1,36 GB
26.9%	org.example.IndexProfiler.measureInsert(SpatialIndex, List)	+2 calls				365,02 MB
25.5%	org.example.IndexProfiler.measureSearchRadius(SpatialIndex, List)					346,11 MB
25.4%	org.example.IndexProfiler.warmup(SpatialIndex, String)	+23 calls				344,14 MB
18.0%	org.example.IndexProfiler.measureSearchNearest(SpatialIndex, List)					243,71 MB
4.2%	org.example.IndexProfiler.generatePoints(int, int)					57,15 MB
< 1%	java.io.PrintStream.println(String)	+7 library calls				1,19 MB
< 1%	java.lang.Thread.run()	+6 library calls				100,5 kB



Выводы: Результаты профайлинга показывают, что основное время выполнения (CPU time) у RTree приходится на рекурсивные вызовы методов findParent и adjustTree, которые многократно вызываются в процессе вставки и последующей корректировки дерева. Это объясняет крайне низкую производительность RTree, зафиксированную в бенчмарках. Основное выделение памяти в RTree происходит при вызове операции с MBR.

В KDTree и OctoTree «горячими» точками являются вычисления расстояний и рекурсивные обходы.

Таким образом, профайлинг показал, что низкая производительность RTree обусловлена комбинацией неэффективных алгоритмов (квадратичное разделение, поиск родителя) и избыточного создания объектов (MBR, временные списки). KDTree и OctoTree, напротив,

демонстрируют минимальные аллокации и более равномерное использование CPU, что коррелирует с их высокими показателями в бенчмарках. Для практического применения в задачах геопоиска предпочтительны OctoTree или KDTree, а RTree требует значительной доработки.